



## **Projeto Prático 02: Jogo Web estilo Atari 2600 (HTML, CSS e JavaScript)**

### **1 - Descrição Geral**

O objetivo deste trabalho é desenvolver, individualmente, um jogo inspirado nos clássicos do Atari 2600, utilizando exclusivamente HTML, CSS e JavaScript puros (sem frameworks).

O aluno poderá escolher:

- a) Qualquer jogo clássico do Atari 2600, implementando ao menos uma fase ou conjunto mínimo de funcionalidades; ou
- b) Desenvolver o jogo Frogger, conforme descrito em: <https://pt.wikipedia.org/wiki/Frogger> (ver esta implementação do Frogger Online <https://happyhopper.org/>)

Um projeto-exemplo foi criado pelo professor como referência de conceitos como grid, movimentação, detecção de colisão, controle de vidas e atualização de interface. Os alunos podem reaproveitar o código visto em aula livremente, mas devem expandi-lo para atender aos requisitos mínimos do projeto.

### **2 - Objetivos de Aprendizagem**

Ao concluir o trabalho, o estudante deverá demonstrar:

- Domínio de estruturas fundamentais de HTML.
- Capacidade de estilização com CSS, incluindo uso de classes, estados e layout responsivo simples.
- Manipulação da árvore DOM com JavaScript.
- Controle de eventos de teclado e/ou mouse.
- Atualização dinâmica da interface com base no estado do jogo.
- Organização mínima do código e clareza na estrutura.

### **3 - Requisitos Mínimos do Projeto**

Para ser considerado apto como trabalho final, o jogo deve conter obrigatoriamente:

#### **3.1 Estrutura e Interface**

- Página HTML única contendo o jogo.
- Uso de CSS para estilização dos elementos:
  - Representação visual do personagem principal;

- Representação dos obstáculos/inimigos;
- Áreas especiais (metas, limites, trilhas, etc).
- Layout com grid, flexbox ou posicionamento absoluto — conforme o estilo escolhido.

### 3.2 Mecânicas do Jogo

- Movimentação do jogador via teclado (teclas de seta ou WASD).
- Detectores de colisão com:
  - obstáculos,
  - limites de tela,
  - zonas proibidas ou perigosas.
- Condição de perda (ex.: perder uma vida ao colidir, reiniciar fase, voltar ao início).
- Condição de vitória (ex.: alcançar o topo no Frogger ou atingir um objetivo no jogo escolhido).
- Contagem de vidas, exibida dinamicamente na tela.
- Sistema de atualização contínua de jogo usando setInterval() ou requestAnimationFrame().

### 3.3 Lógica Implementada em JavaScript

- Código organizado em funções.
- Atualização de classes CSS para representar estados (ex.: azul/vermelho no exemplo).
- Criação e manipulação de elementos DOM quando necessário.
- Armazenamento e atualização de estado do jogo (posições, vidas, direção, etc.).

## 4 - Requisitos Adicionais para Nota Máxima

Para diferenciação de níveis de complexidade, trabalhos que atingirem avaliação máxima devem incluir pelo menos dois dos itens abaixo:

- Efeitos sonoros simples (colisões, vitória, derrota).
- Layout parcialmente responsivo (ajuste mínimo para telas maiores).
- Sistema de pontos ou fases adicionais.
- Tela inicial e tela de game over personalizadas.
- Sprites simples feitos com CSS ou imagens otimizadas.

## 5 - Entrega

A entrega deverá conter todos os arquivos de código fonte HTML, CSS, JS e imagens organizados

em diretórios adequadamente, com o arquivo index.html na raiz do projeto. A entrega deve ser feita através de commit no repositório Git do aluno, criado através do link do GitHub Classroom fornecido pelo professor.

O repositório deve conter também, na raiz, um arquivo README.md com nome do aluno, jogo escolhido, controles (orientações básicas para jogar) e descrição das mecânicas implementadas.

## 6 - Critérios de Avaliação

| <b>Critério</b>                                      | <b>Peso</b>      |
|------------------------------------------------------|------------------|
| Estrutura HTML adequada                              | 2,0              |
| Estilização CSS e organização visual                 | 2,0              |
| Implementação da lógica do jogo em JS                | 3,0              |
| Funcionalidade completa do jogo (requisitos mínimos) | 2,0              |
| Requisitos avançados / capricho / clareza do código  | 1,0              |
| <b>Total</b>                                         | <b>10 pontos</b> |

## 7 - Observações Importantes

O aluno não precisa reproduzir fielmente a dificuldade ou física do jogo original, apenas suas ideias principais.

- O projeto deve ser originalmente desenvolvido pelo estudante.
- Códigos externos podem ser usados como inspiração, mas não podem ser copiados integralmente.
- O professor poderá solicitar explicação oral do funcionamento para validar autoria.

### **Plágio não é tolerado**

Você deve ser o único(a) responsável por fazer a entrega para essa atividade. Todo o código ou texto deverá ser produzido exclusivamente por você, exceto trechos de códigos que possam ter sido fornecidos como parte do enunciado.

Você pode discutir com outros estudantes com o intuito de esclarecer pontos, isso é até incentivado, porém você não poderá copiar trechos de códigos, textos ou soluções de qualquer fonte (e.g. colegas da mesma turma ou curso, repositórios de códigos na Internet ou soluções providas por serviços como Copilot e ChatGPT).